CS 5402: Introduction to Data Mining

SP2026

(Lecture - 19)

Transformer

Dr. Huiyuan Yang

Department of Computer Science
Missouri S&T

https://hyang428.github.io

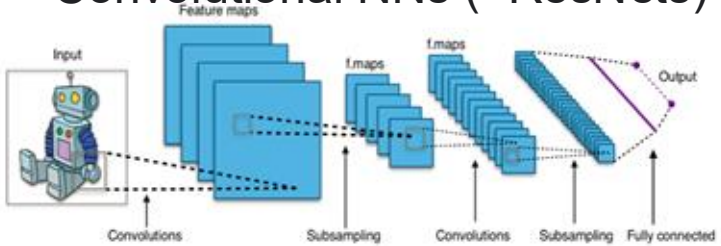
MISSOURI S&T

In-Class Participation Question (Q9)

- **T / F**: SSL creates supervisory signals directly from the data itself.
- **T / F**: SSL representations are typically less useful for downstream tasks than supervised learning.
- **T / F**: The performance on a pretext task always directly reflects downstream task performance.
- **T / F**: Contrastive learning requires both positive and negative sample pairs.
- **T / F**: BYOL requires negative samples to avoid collapse.

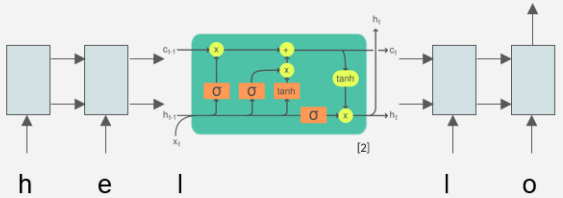
Computer Vision

Convolutional NNs (+ResNets)



Natural Lang. Proc.

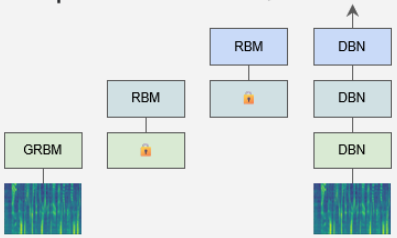
Recurrent NNs (+LSTMs)



Different architectures for different tasks & modality

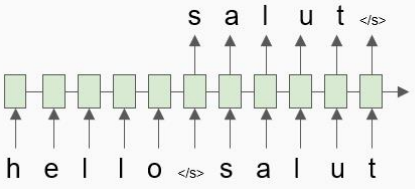
Speech

Deep Belief Nets (+non-DL)



Translation

Seq2Seq



RL

BC/GAIL

Algorithm 1 Generative adversarial imitation learning

1: **Input:** Expert trajectories $\tau_E \sim \pi_E$, initial policy and discriminator parameters θ_0, w_0
2: **for** $i = 0, 1, 2, \ldots$ **do**
3: Sample trajectories $\tau_i \sim \pi_{\theta_i}$
4: Update the discriminator parameters from w_i to w_{i+1} with the gradient

$$\hat{\mathbb{E}}_{\tau_i}[\nabla_w \log(D_w(s,a))] + \hat{\mathbb{E}}_{\tau_E}[\nabla_w \log(1 - D_w(s,a))] \quad (17)$$

5: Take a policy step from θ_i to θ_{i+1}, using the TRPO rule with cost function $\log(D_{w_{i+1}}(s,a))$. Specifically, take a KL-constrained natural gradient step with

$$\hat{\mathbb{E}}_{\tau_i}[\nabla_\theta \log \pi_\theta(a|s) Q(s,a)] - \lambda \nabla_\theta H(\pi_\theta), \quad \text{where } Q(\bar{s},\bar{a}) = \hat{\mathbb{E}}_{\tau_i}[\log(D_{w_{i+1}}(s,a)) \,|\, s_0 = \bar{s}, a_0 = \bar{a}] \quad (18)$$

6: **end for**

The Vanilla Transformer

Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com

Noam Shazeer* Google Brain noam@google.com

Niki Parmar* Google Research nikip@google.com

Jakob Uszkoreit* Google Research usz@google.com

Llion Jones* Google Research llion@google.com

Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu

Łukasz Kaiser* Google Brain lukaszkaiser@google.com

Illia Polosukhin* ‡ illia.polosukhin@gmail.com

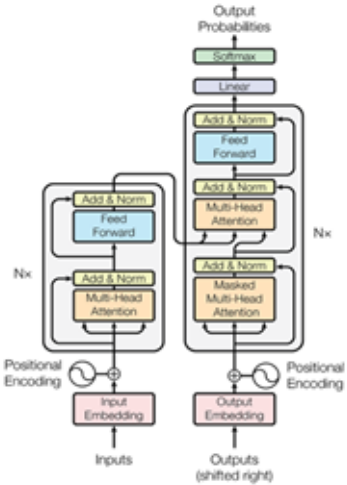


Figure 1: The Transformer - model architecture.

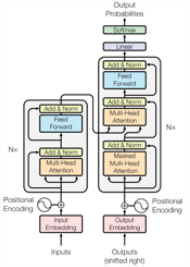
NeurIPS 2026

https://papers.nips.cc › paper › 7181-attention-is-all-yo...

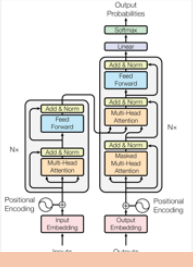
Attention is All you Need

by A Vaswani · 2017 · Cited by 240065 — We propose a novel, simple network architecture based solely onan attention mechanism, dispensing with recurrence and convolutions entirely. Read more

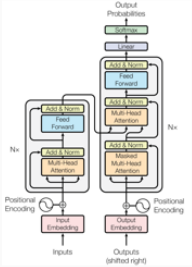
Computer Vision



Natural Lang. Proc.

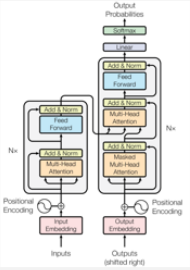


Reinf. Learning

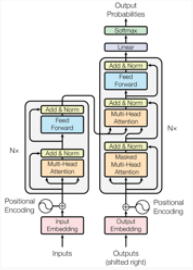


One architecture for all

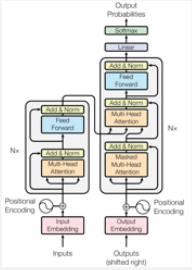
Speech



Translation

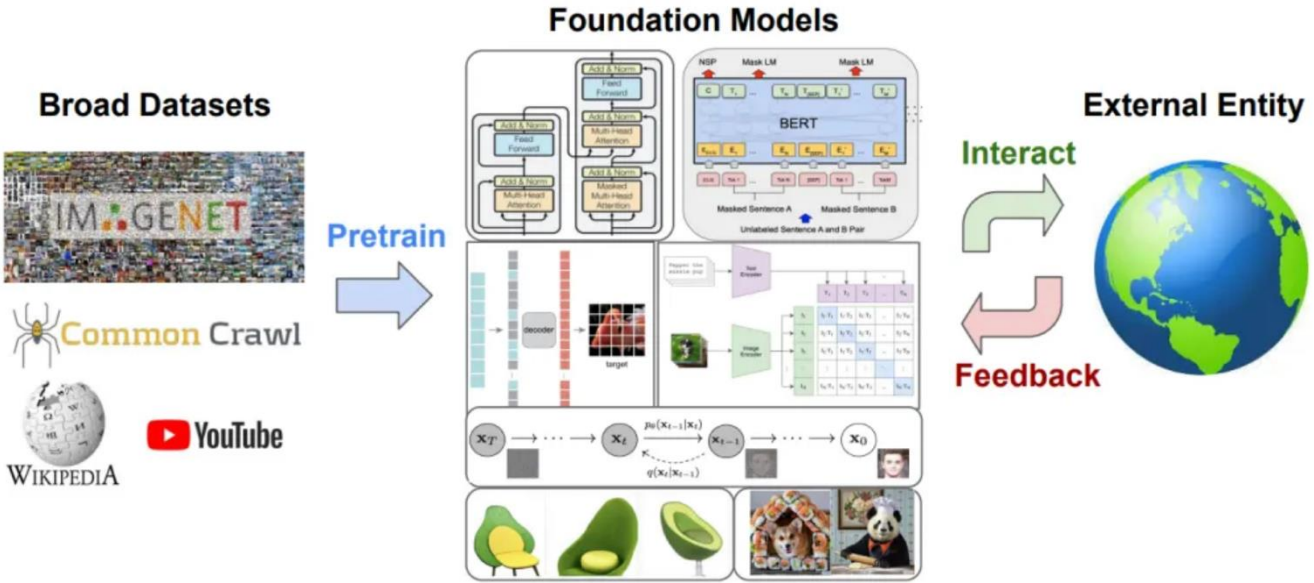


Graphs/Science



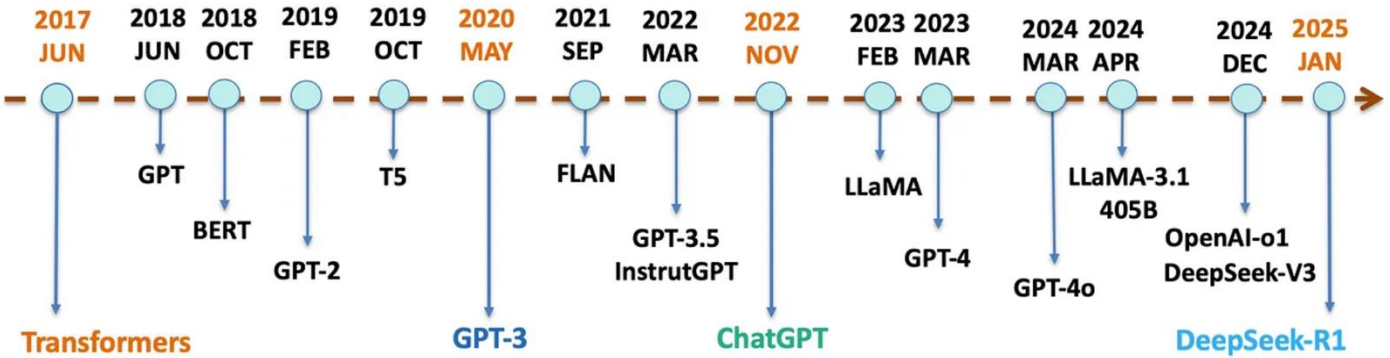
Transformer image source: "Attention Is All You Need" paper

Foundation Models: **One Model, Many Tasks**



You train **one giant model** on a massive amount of broad data. This model (Foundation Model) learns the general patterns of the world (grammar, logic, visual shapes, etc.). Once this "foundation" is built, you can slightly tweak (fine-tune) it to do almost anything.

History of Transformers & Applications



https://medium.com/@lmpo/a-brief-history-of-lmms-from-transformers-2017-to-deepseek-r1-2025-dae75dd3f59a

Overview of the transformer model

Sequence-to-sequence; Encoder-Decoder;

for the task of machine translation:

- ❑ Input/Output Embedding
- ❑ Positional Encoding
- ❑ Self-Attention
  - ❑ Multi-Head Attention
- ❑ Feed Forward Layer
- ❑ Add & Norm Layer
- ❑ Creating Masks

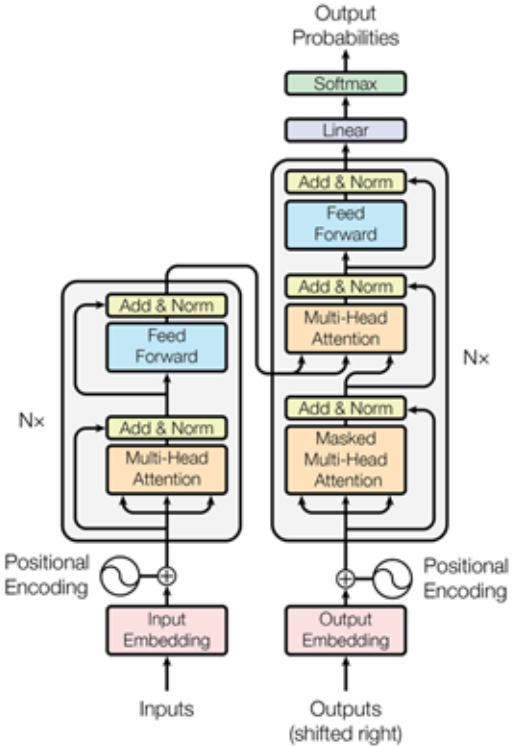
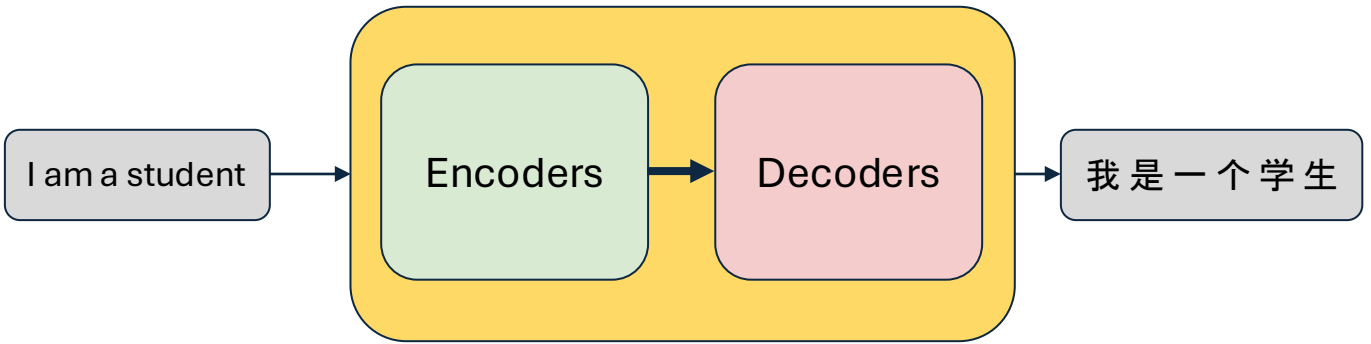


Figure 1: The Transformer - model architecture.

A high level look (1)

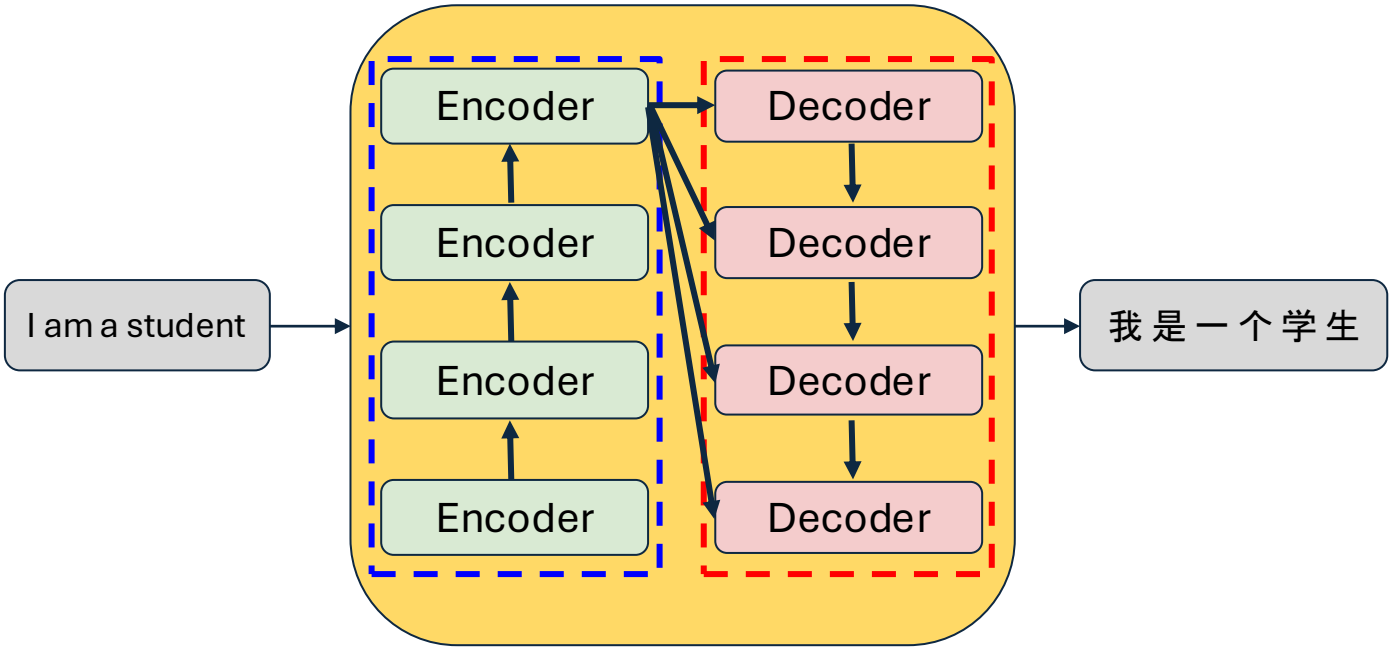


A high level look (2)

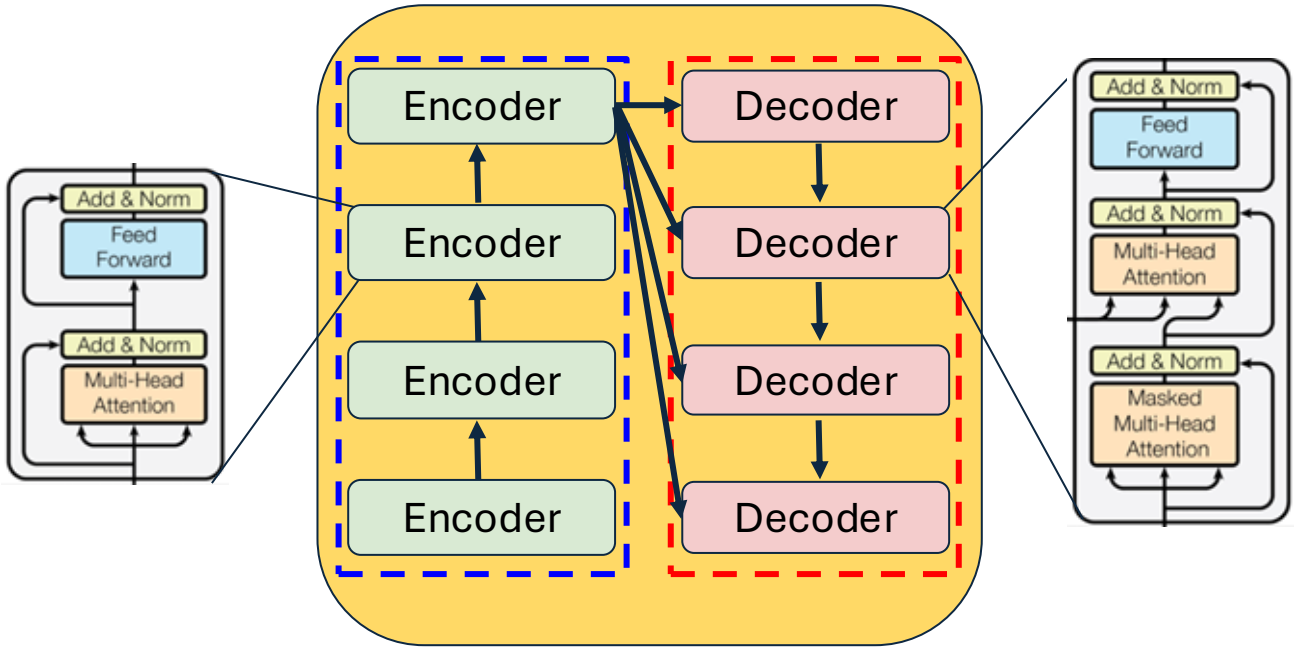


Transformer

A high level look (3)



A high level look (4)



Transformer Architecture

- ❑ Input/Output Embedding
- ❑ Positional Encoding
- ❑ Self-Attention
  - ❑ Multi-Head Attention
- ❑ Feed Forward Layer
- ❑ Add & Norm Layer
- ❑ Creating Masks

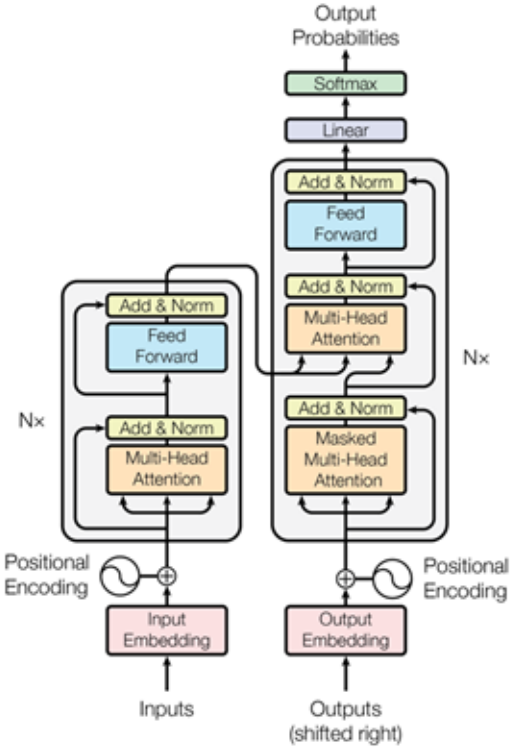
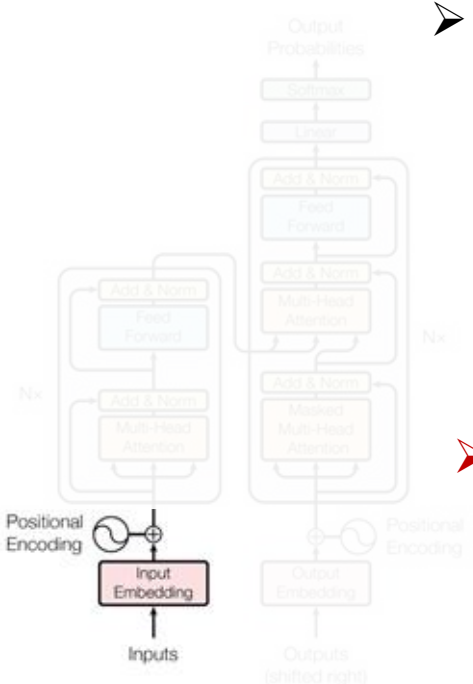


Figure 1: The Transformer - model architecture.

Input/Output Embedding (1)



- Why do we need embeddings?

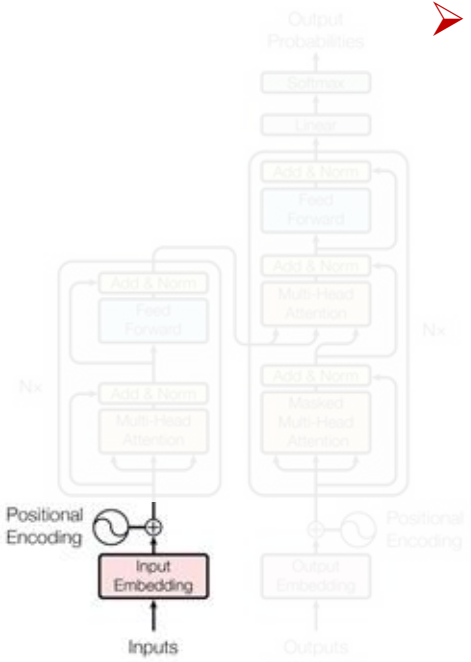
Computers only understand numbers, not words like:

"I", "love", "dog"

Therefore, we must convert words into numerical representations.

- But, how?

Input/Output Embedding (2)



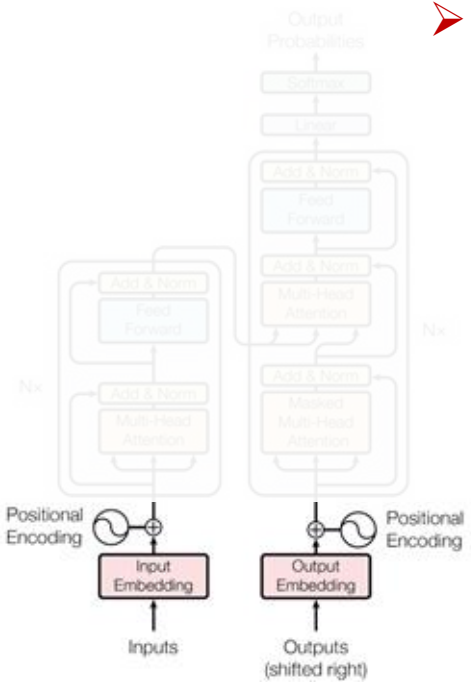
- Use IDs or One-Hot encoding?

Word	ID
I	1
love	2
dog	3
cat	4

Problems:

- Does "2" being closer to "3" than "100" mean anything?
- "cat" and "dog" are semantically similar, but IDs do not reflect this .

Input/Output Embedding (3)



➢ Core idea of embedding

❖ Represent each word as a vector:

"cat" → $[0.2, \ldots, 0.1, 0.9]$

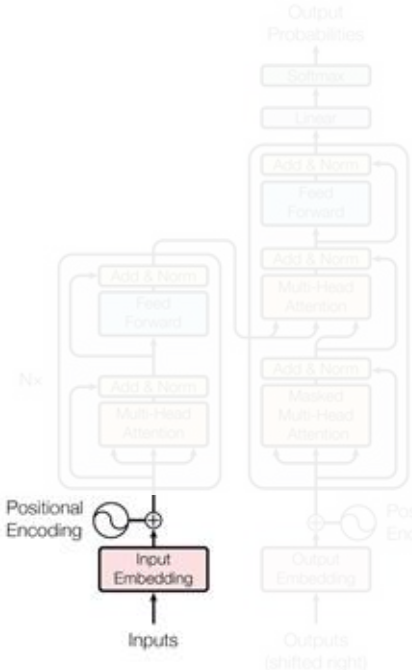
"dog" → $[0.25, \ldots, 0.05, 0.85]$

Result:

✓ "cat" and "dog" → vectors are close

✓ "cat" and "car" → vectors are far apart

Input/Output Embedding (4)



- **Input (Tokenization and) Embedding**

Input text is first split into pieces. Can be characters, word, "tokens":

"The detective investigated the event" -> [The] [detective] [investigated] [the] [event]

Tokens are indices into the "vocabulary":

[The] [detective] [investigated] [the] [event] -> [3 721 68 1337 42]

Each vocab entry corresponds to a learned d_{model}-dimensional vector.

[3 721 68 1337 42] -> [[0.123, -5.234, ...], [...], [...], [...], [...]]

- **Positional Encoding**

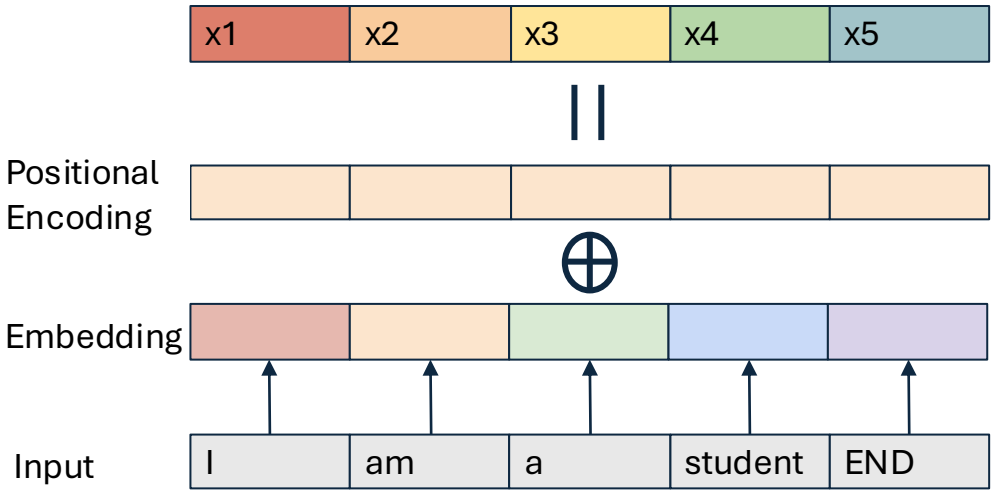
Remember attention is permutation invariant, but language is not!

("The mouse ate the cat" vs "The cat ate the mouse")

Need to encode position of each word; just add something.

[The_] + 10 [detective_] + 20 [invest] + 30 …

Input/Output Embedding (5)



Positional Encoding (1)

Since the self-attention mechanism doesn't build in order information, we need to encoder the order information (position) in the sentence.

("The mouse ate the cat" vs "The cat ate the mouse")

(Both Convolution & Recurrent models process words in a particular order, but the transformer doesn't (everything happens in parallel)). We want to inject some temporal information so that the model knows the relationships between words based on their context.

$$PE_{(pos,2i)} = sin(pos/10000^{2i/d_{\text{model}}})$$
$$PE_{(pos,2i+1)} = cos(pos/10000^{2i/d_{\text{model}}})$$

- pos: position of an object in the input sequence
- d_{model}: Dimension of the output embedding space
- i: Used for mapping to column indices $0 \leq i \leq d_{model}/2$, with s single value of I maps to both since and cosine functions.
- 10000: empirically selected constant, used to capture both short- and long-term relationship.

Example: Positional Encoding

$$PE(pos, 2i) = \sin\left(\frac{pos}{n^{2i/d}}\right)$$

$$PE(pos, 2i+1) = \cos\left(\frac{pos}{n^{2i/d}}\right)$$

$d=4$

$n=100$

$0 \le i < \frac{d}{2}$

$i=0 \Rightarrow PE(pos, 0) = \sin\left(\frac{pos}{1}\right)$

$PE(pos, 1) = \cos\left(\frac{pos}{1}\right)$

$i=1 \Rightarrow PE(pos, 2) = \sin\left(\frac{pos}{10}\right)$

$PE(pos, 3) = \cos\left(\frac{pos}{10}\right)$

Positional Encoding Matrix with d=4, n=100

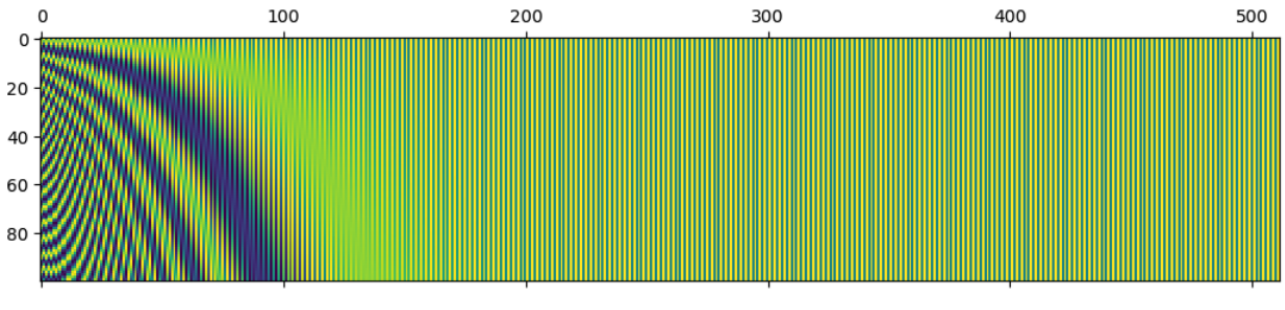
Sequence	Index of token, pos	i=0	i=0	i=1	i=1
I	0	P_{00}=sin(0) = 0	P_{01}=cos(0) = 1	P_{02}=sin(0) = 0	P_{03}=cos(0) = 1
am	1	P_{10}=sin(1/1) = 0.84	P_{11}=cos(1/1) = 0.54	P_{12}=sin(1/10) = 0.10	P_{13}=cos(1/10) = 1.0
a	2	P_{20}=sin(2/1) = 0.91	P_{21}=cos(2/1) = -0.42	P_{22}=sin(2/10) = 0.20	P_{23}=cos(2/10) = 0.98
Robot	3	P_{30}=sin(3/1) = 0.14	P_{31}=cos(3/1) = -0.99	P_{32}=sin(3/10) = 0.30	P_{33}=cos(3/10) = 0.96

Positional Encoding Matrix for the sequence 'I am a robot'

Image credits to: https://machinelearningmastery.com/a-gentle-introduction-to-positional-encoding-in-transformer-models-part-1/

Positional Encoding (2)

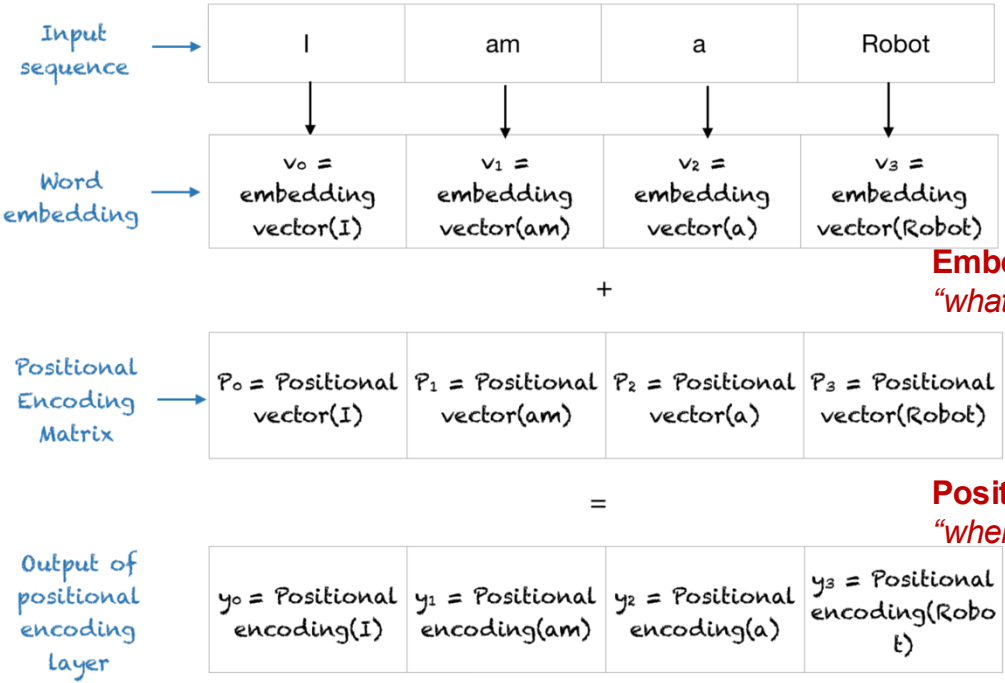
Positional Encoding injects sequence order information into the Transformer, enabling the model to understand the relationships between words in terms of their positions.



The positional encoding matrix for n=10,000, d=512, sequence length=100

Image credits to: https://machinelearningmastery.com/a-gentle-introduction-to-positional-encoding-in-transformer-models-part-1/

Positional Encoding (3)



Embedding tells you:
"what the word is"

Positional encoding tells you:
"where the word is"

Image credits to: https://machinelearningmastery.com/a-gentle-introduction-to-positional-encoding-in-transformer-models-part-1/

Transformer Architecture: Self-Attention

- ❑ Input/Output Embedding
- ❑ Positional Encoding
- ❑ Self-Attention
  - ❑ Multi-Head Attention
- ❑ Feed Forward Layer
- ❑ Add & Norm Layer
- ❑ Creating Masks

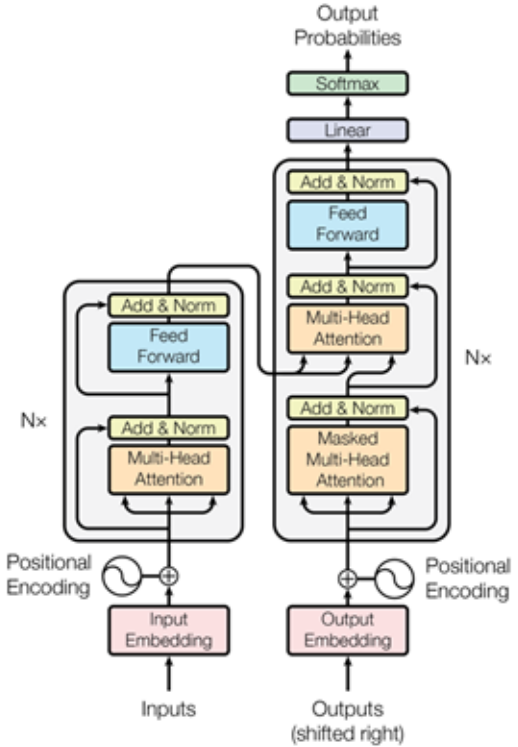
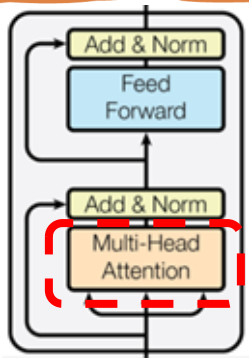


Figure 1: The Transformer - model architecture.

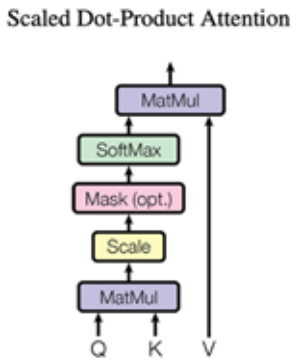
Self-Attention (1)

- Self-attention is the core building block of transformer (Multi-Headed Self-Attention).
- Each token can "*look around*" the whole input, and decide how to update its representation based on what it sees.



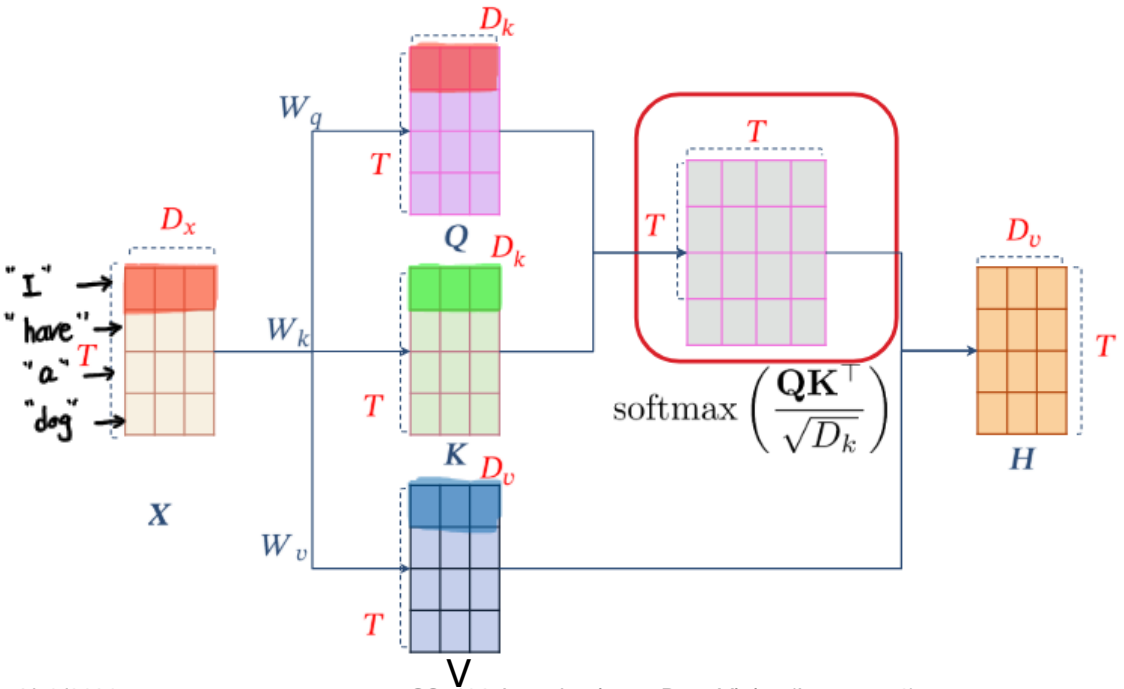
$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

V: Key
K: Value
Q: Query



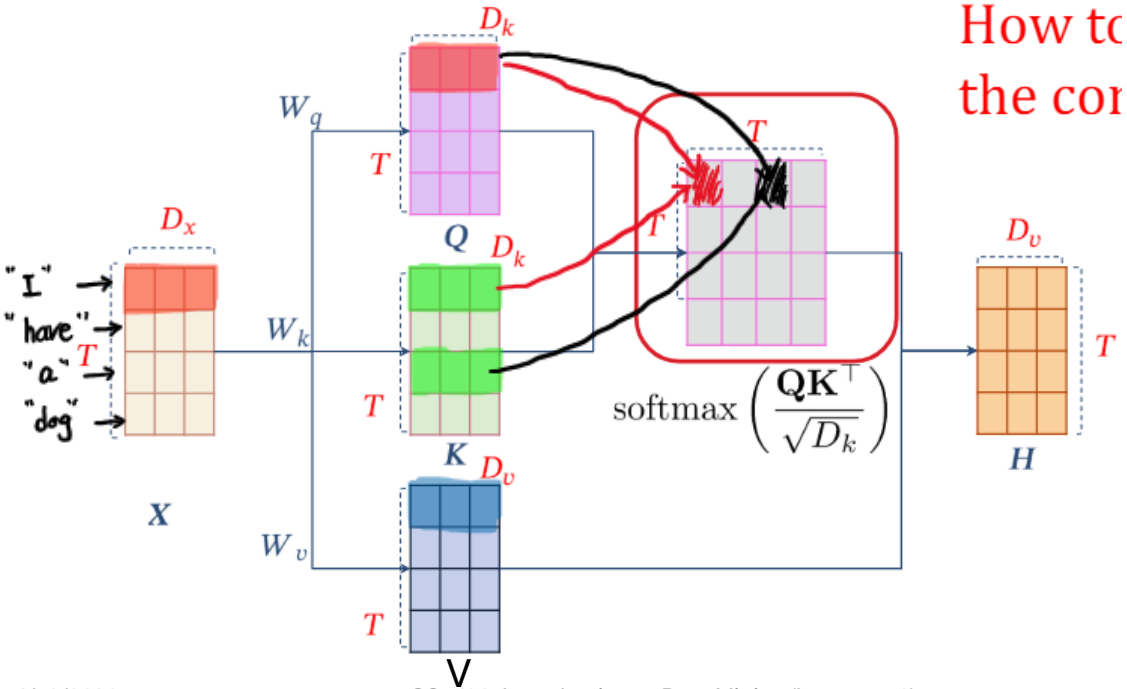
Self-Attention (2)

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$



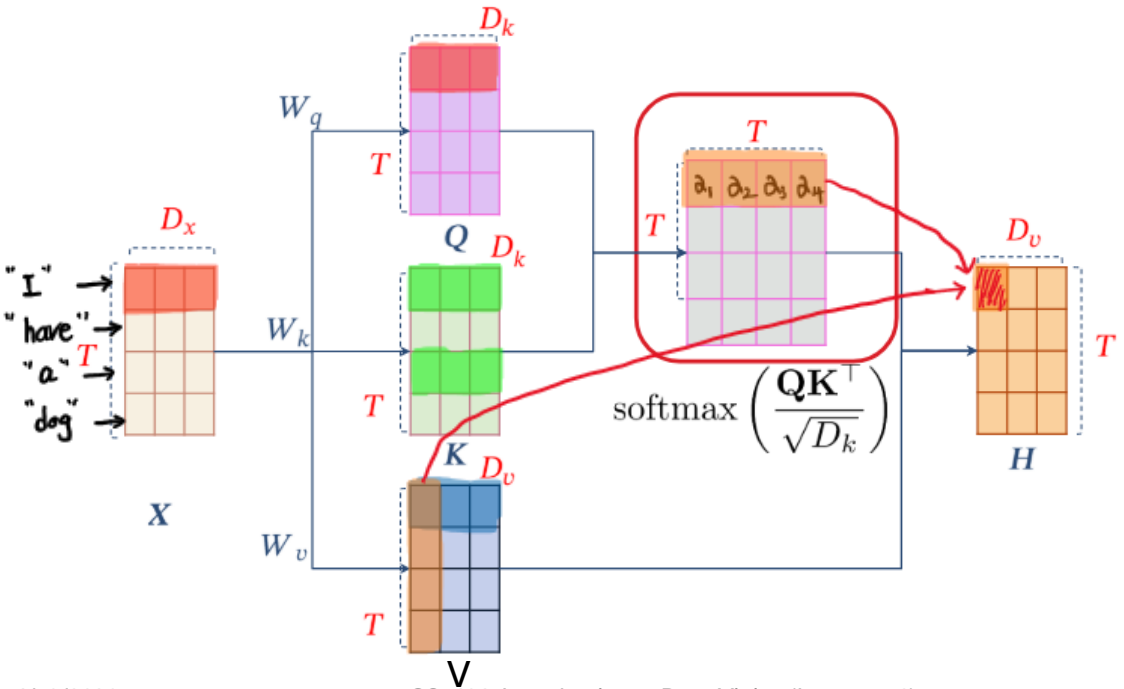
Self-Attention (3)

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$



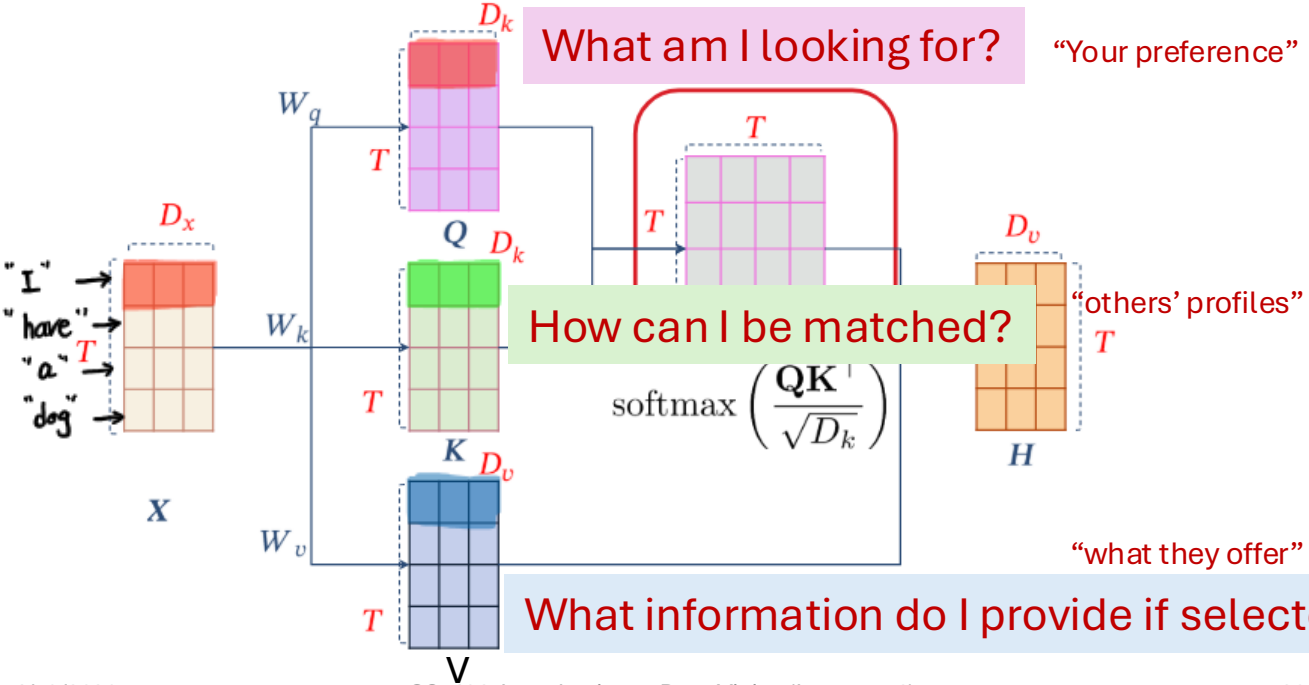
Self-Attention (4)

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$



Self-Attention: why do we need W_q, W_k, W_v

W_q, W_k, W_v are used to project the same input into **different semantic spaces**, allowing each token to play different roles: *query*, *key*, and *value*.



Self-Attention: Example (1)

Self-attention for individual token:

- Step 1: For each word x_i , calculate its **query**, **key**, and **value**.

$$q_i = W^Q x_i \quad k_i = W^K x_i \quad v_i = W^V x_i$$

- Step 2: Calculate attention score between **query** and **keys**.

$$e_{ij} = q_i \cdot k_j$$

- Step 3: Take the softmax to normalize attention scores.

$$\alpha_{ij} = softmax(e_{ij}) = \frac{exp(e_{ij})}{\sum_k exp(e_{ik})}$$

- Step 4: Take a weighted sum of **values**.

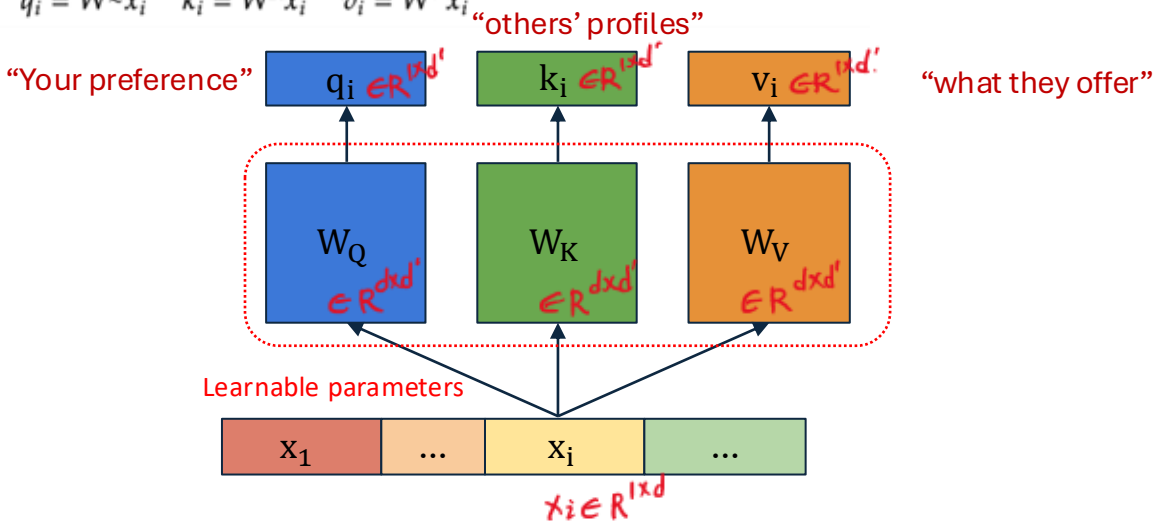
$$Output_i = \sum_j \alpha_{ij} v_j$$

Credits to CS224N/Ling284 Stanford

Self-Attention: Example (2)

- Step 1: For each word x_i, calculate its **query**, **key**, and **value**.

$$q_i = W^Q x_i \quad k_i = W^K x_i \quad v_i = W^V x_i$$



Self-Attention: Example (3)

- Step 1: For each word x_i , calculate its **query**, **key**, and **value**.

$$q_i = W^Q x_i \quad k_i = W^K x_i \quad v_i = W^V x_i$$

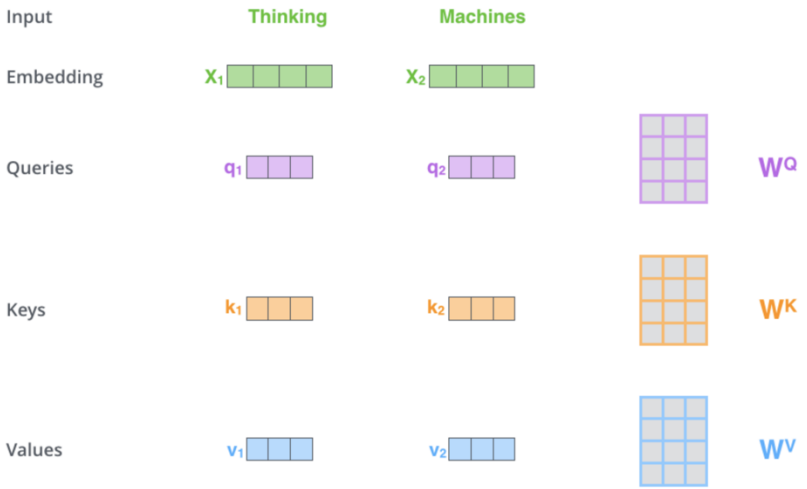


Image credits to https://jalammar.github.io/illustrated-transformer/

Self-Attention: Example (4)

- Step 2: Calculate attention score between **query** and **keys.**

$$e_{ij} = q_i \cdot k_j$$

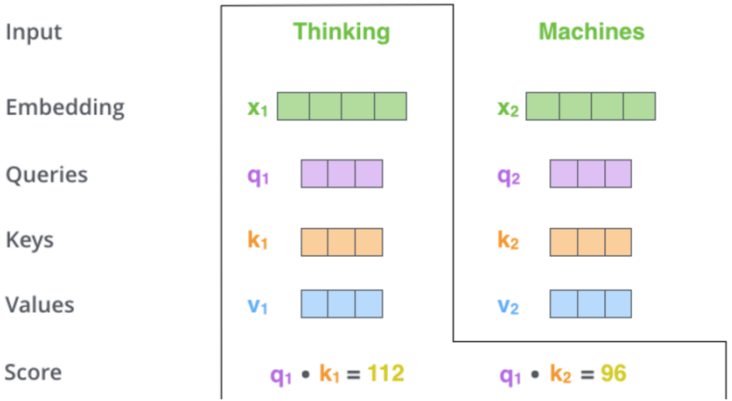


Image credits to https://jalammar.github.io/illustrated-transformer/

Self-Attention: Example (5)

- Step 3: Take the softmax to normalize attention scores.

$$\alpha_{ij} = softmax(e_{ij}) = \frac{exp(e_{ij})}{\sum_k exp(e_{ik})}$$

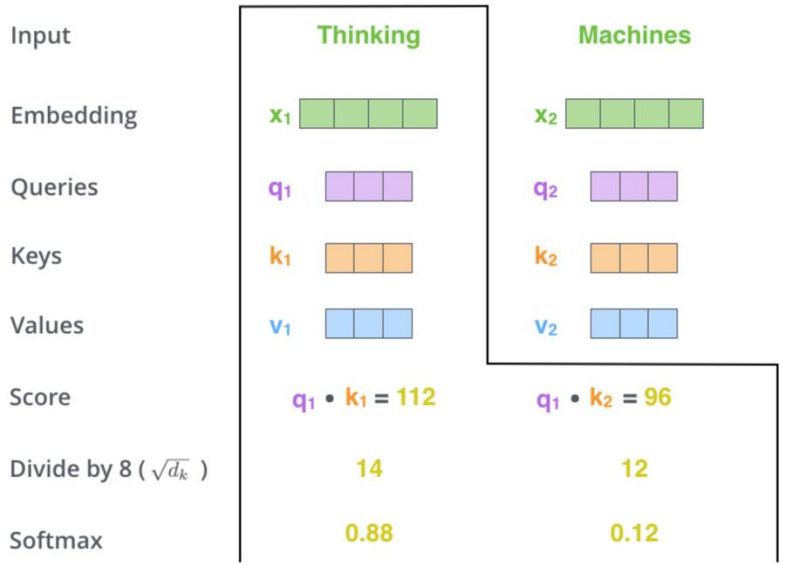


Image credits to https://jalammar.github.io/illustrated-transformer/

Self-Attention: Example (6)

- Step 4: Take a weighted sum of **values**.

$$Output_i = \sum_j \alpha_{ij} v_j$$

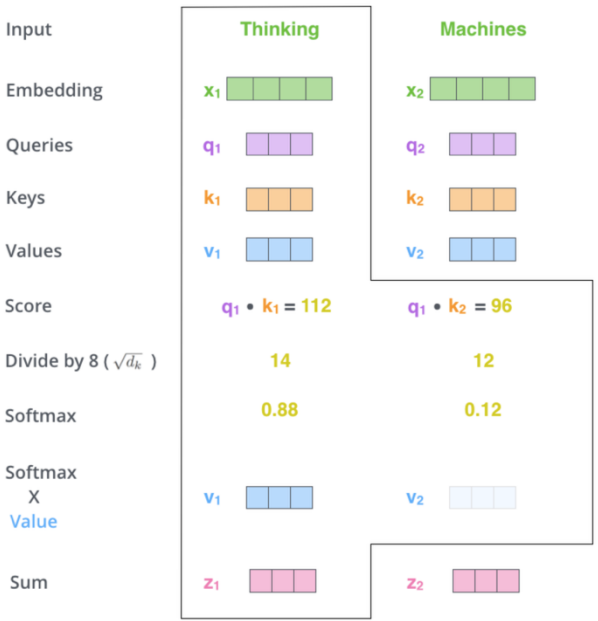
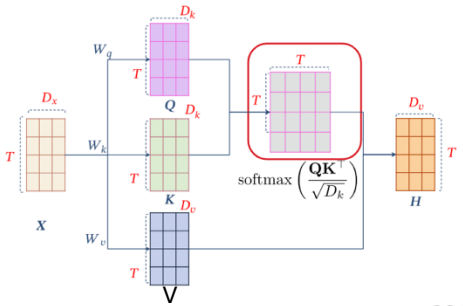
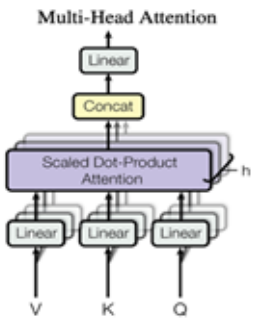


Image credits to https://jalammar.github.io/illustrated-transformer/

Multi-Head Attention (1)

The animal didn't cross the street because it was too tired.





What different heads can learn

❖ **Head 1: Coreference (long-range dependency)**

Focuses on linking **"it"** → **"animal"**

Captures semantic meaning and reference

❖ **Head 2: Syntactic structure**

Focuses on relationships like:

subject–verb: "animal → didn't cross"

Learns grammatical structure

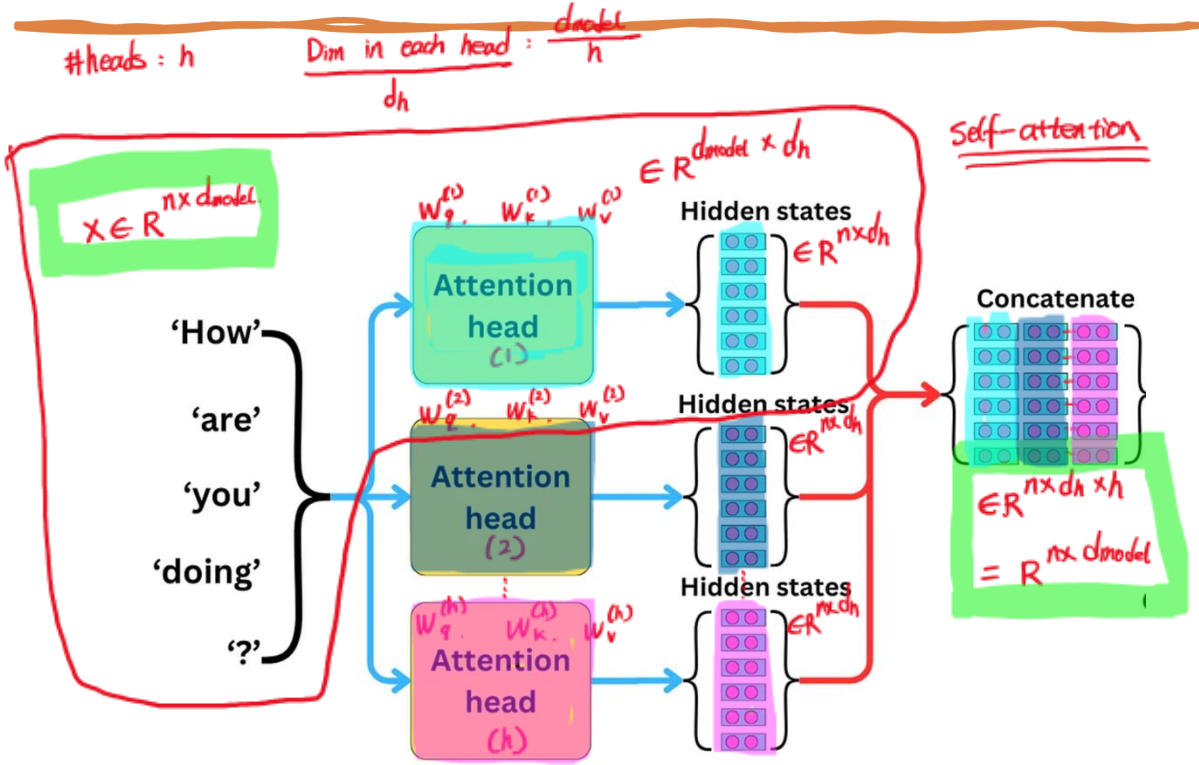
❖ **Head 3: Local context**

Focuses on nearby words:

"too → tired"

Captures short-range dependencies

Multi-Head Attention (2)



Transformer Architecture: FFN

- ❑ Input/Output Embedding
- ❑ Positional Encoding
- ❑ Self-Attention
  - ❑ Multi-Head Attention
- ❑ Feed Forward Layer
- ❑ Add & Norm Layer
- ❑ Creating Masks

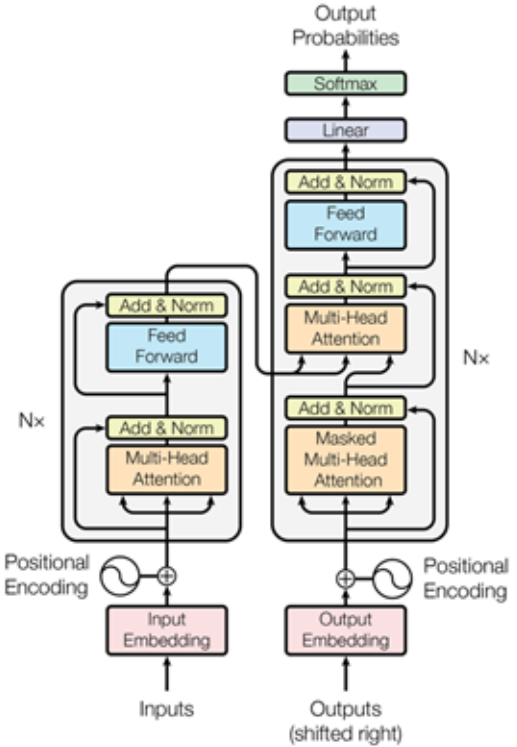
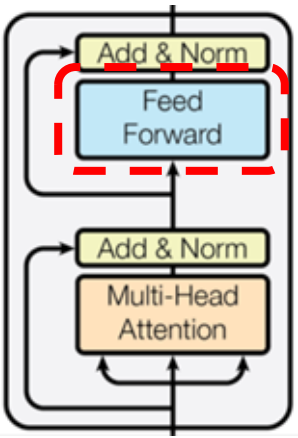


Figure 1: The Transformer - model architecture.

Feed Forward Layer

This layer just consists of two linear operations, with a relu and dropout operation in between them.




```
class FeedForward(nn.Module):
    def __init__(self, d_model, d_ff=2048, dropout = 0.1):
        super().__init__()
        # We set d_ff as a default to 2048
        self.linear_1 = nn.Linear(d_model, d_ff)
        self.dropout = nn.Dropout(dropout)
        self.linear_2 = nn.Linear(d_ff, d_model)
    def forward(self, x):
        x = self.dropout(F.relu(self.linear_1(x)))
        x = self.linear_2(x)
        return x
```

Transformer Architecture: Add & Norm

- ❑ Input/Output Embedding
- ❑ Positional Encoding
- ❑ Self-Attention
  - ❑ Multi-Head Attention
- ❑ Feed Forward Layer
- ❑ Add & Norm Layer
- ❑ Creating Masks

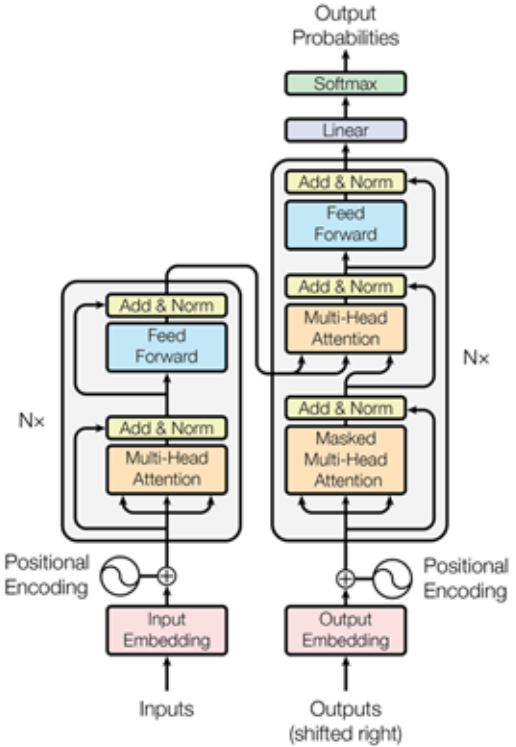
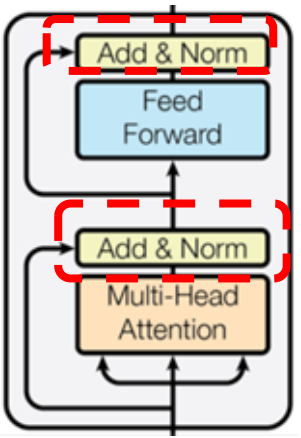


Figure 1: The Transformer - model architecture.

Add & Norm Layer

Used twice in each Encoder, and three times in each Decoder, including:

- Residual connection
  - Similar structure proposed in ResNet, ResNets help in tackling the vanishing gradient problem using identity mapping. This prevents the network from "forgetting" or distorting important information as it is processed by many layers
- Layer normalization
  - Very important in deep models, as it prevents the range of values in the layers changing too much, meaning the model trains faster and has better ability to generalize.



Transformer Architecture: Masking

- ❑ Input/Output Embedding
- ❑ Positional Encoding
- ❑ Self-Attention
  - ❑ Multi-Head Attention
- ❑ Feed Forward Layer
- ❑ Add & Norm Layer
- ❑ Creating Masks

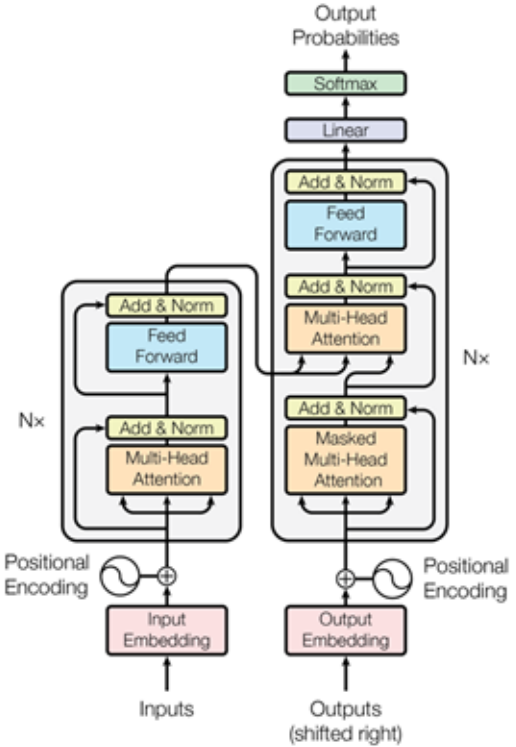


Figure 1: The Transformer - model architecture.

Creating Masks (1)

Masking plays an important role in the transformer, and it serves two purposes:

I. In the encoder: to zero attention outputs wherever there is just padding in the input sentences.

Mask	1	1	1	1	1	0	0
Input	I	am	a	student	END	N/A	N/A

Mask	1	1	1	1	0	0	0
Input	Good	Morning	!	END	N/A	N/A	N/A

Creating Masks (2)

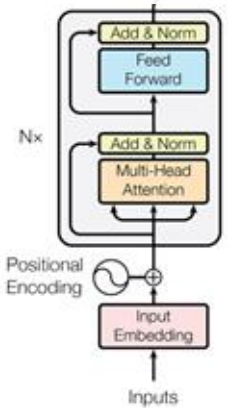
Masking plays an important role in the transformer, and it serves two purposes:

I. In the encoder: to zero attention outputs wherever there is just padding in the input sentences.

II. In the decoder: to prevent the decoder 'peaking' ahead at the rest of the translated sentence when predicting the next word.



Transformer Architecture: Encoder



Encoding / Encoder

- Since input and output shapes are identical, we can stack N such blocks.

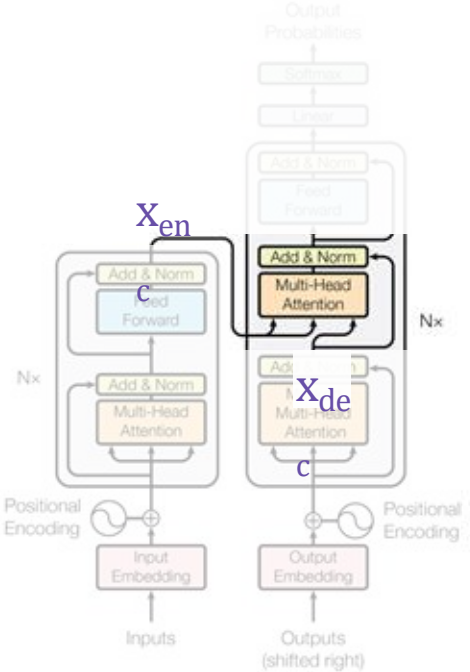
  Typically, N=6 ("base"), N=12 ("large") or more.

- Encoder output is a "heavily processed" (think: "high level, contextualized") version of the input tokens, i.e. a sequence.

- This has nothing to do with the requested output yet (think: translation). That comes with the decoder.

ChatGPT used 80~120 transformer layers

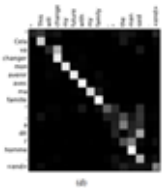
Transformer Architecture: Decoder (1)



"Cross" attention

Each decoded token can "look at" the encoder's output:

$$\text{Attn}(q=W_q x_{dec},\ k=W_k x_{enc},\ v=W_v x_{enc})$$



This is the same as in the 2014 paper.

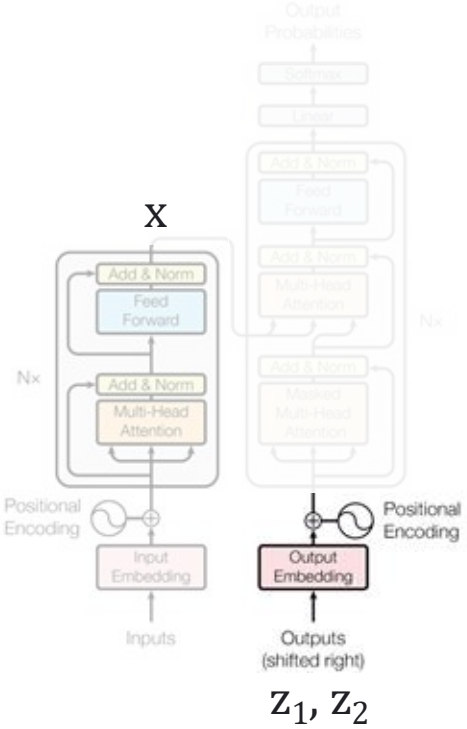
This is where x in $p(z_3|z_2,z_1,x)$ comes from.

Because self-attention is so widely used, people have started just calling it "attention".

Hence, we now often need to explicitly call this "cross attention".

Transformer Architecture: Decoder (2)

$p(z_3|z_2,z_1,x)$



Decoding / the Decoder (alternatively Generating / the Generator)

What we want to model: p(z|x)

for example, in translation: p(z | "the detective investigated") **∀z**

Seems impossible at first, but we can *exactly* decompose into tokens:

$p(z|x) = p(z_1|x)\ p(z_2|z_1,x)\ p(z_3|z_2,z_1,x)\ldots$

Meaning, we can generate the answer one token at a time.

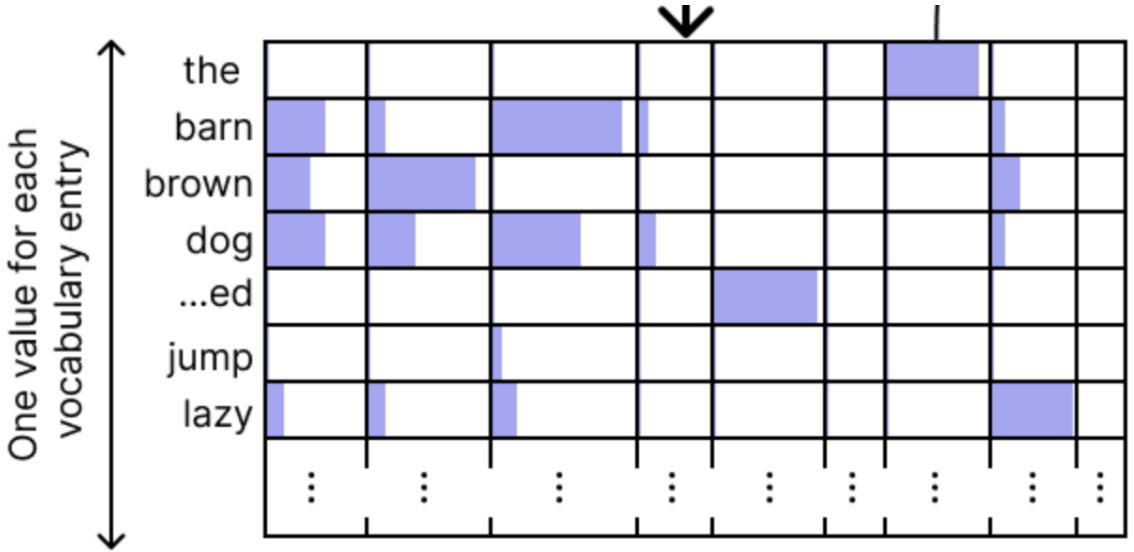
Each p(.) is a full pass through the model.

For generating $p(z_3|z_2,z_1,x)$:

- x comes from the encoder,
- z_1, z_2 is what we have predicted so far, goes into the decoder.

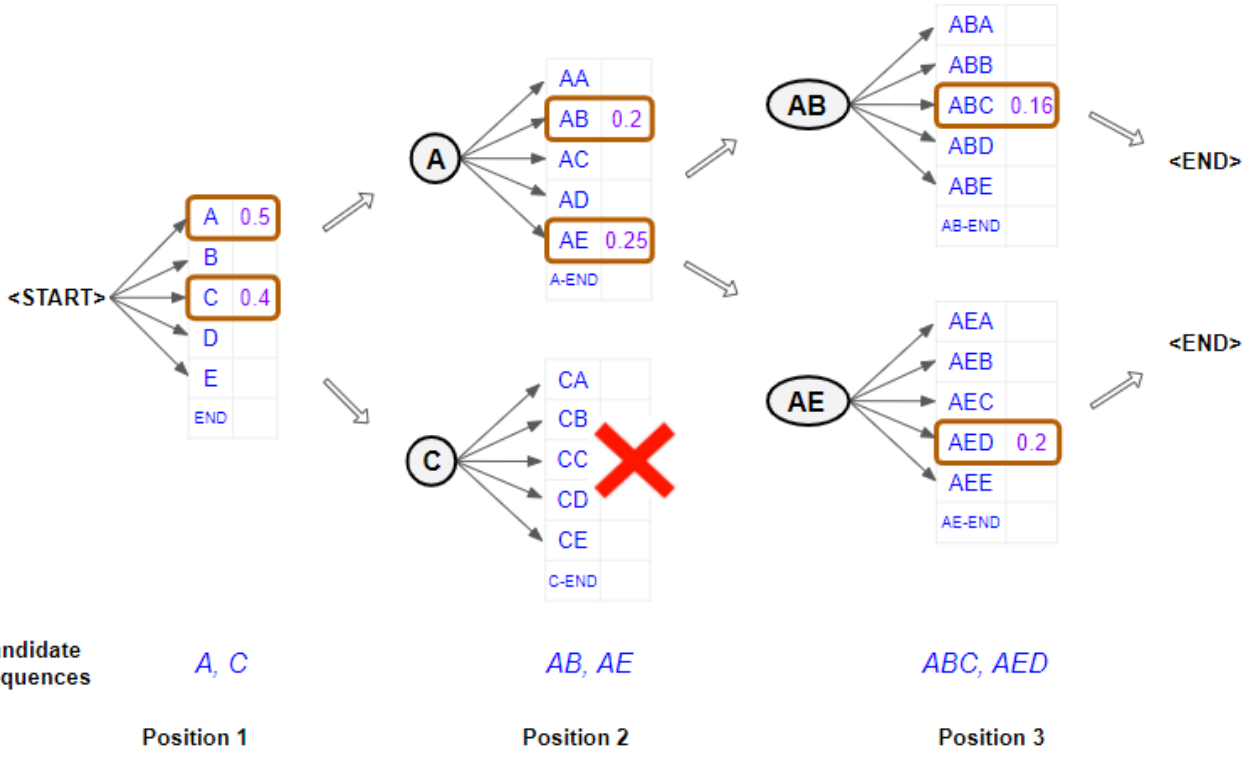
Once we have p(z|x) we still need to actually sample a sentence such as "le détective a enquêté". Many strategies: greedy, beam-search, …

Transformer Architecture: Decoder (3)



Greedy strategy

Transformer Architecture: Decoder (4)



Beam-search

Why transformer works (powerful)

- **Global Context Modeling**
  Each token attends to *all others* → captures long-range dependencies
- **Parallel Computation**
  Processes entire sequence at once → faster and scalable training
- **Multi-Head Attention**
  Learns different types of relationships (syntax, semantics, structure)
- **Rich Representations**
  Deep stacking + nonlinearity → highly expressive features
- **Flexible & General**
  Works across text, vision, speech, and multimodal tasks

Transformers are powerful because they model all relationships in a sequence simultaneously, efficiently, and at scale.

Impact & Applications

Transformer in Practice

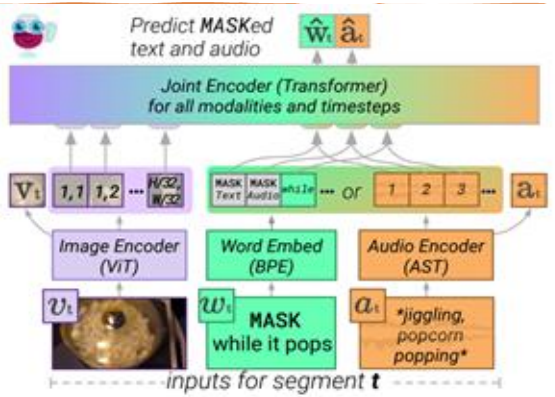
- Widely used in:
  - **Natural Language Processing** (e.g., BERT, GPT)
  - **Computer Vision** (e.g., Vision Transformer)
  - **Time Series Analysis** (e.g., PatchTST)
  - **Multimodal** (e.g., GPT-4, Gemini)

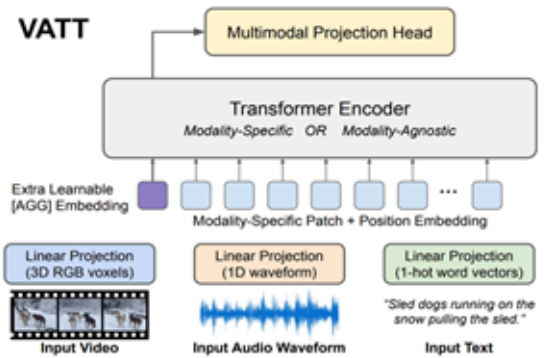
Key Role: Foundation of modern AI systems

Anything you can tokenize, you can feed to Transformer

Tokenize different modalities each in their own way (some kind of "patching"), and send them all jointly into a Transformer...

Seems to just work...
Currently an explosion of works doing this!





Images from:
[1] LIMoE by B Mustafa, C Riquelme, J Puigcerver, R Jenatton, N Houlsby
[2] MERLOT Reserve by R Zellers, J Lu, X Lu, Y Yu, Y Zhao, M Salehi, A Kusupati, J Hessel, A Farhadi, Y Choi
[3] VATT by H Akbari, L Yuan, R Qian, W-H Chuang, S-F Chang, Y Cui, B Gong

After-Class Practice Question

Q1. Explain why self-attention is permutation-invariant. (a) why does this create a problem? (b) how does positional encoding resolve it?

Q2. Compare self-attention vs RNN/LSTM in terms of: dependency modeling, computational complexity and parallelization.

Q3. In the Transformer encoder, why are residual connections + layer normalization necessary? (a) what problems do they solve? (b) what would happen if we remove them?

Q4. Why does the Transformer decoder use masking? (a) what would happen if masking is removed during training? (b) how does this relate to inference?

Q5. Implement the self-attention module.